

Assignment:16.2

Name: Deekshith Goud

Roll no.: 2303A52104

Course: Ai coding

Batch:33

Task Description-1: (Design Database Schema for Student Management System)

Instructions:

- Using AI tools, generate CREATE TABLE statements for a Student Management Database.
- Tables: Students, Courses, Enrollments.
- Include primary keys, foreign keys, and suitable data types. Ensure the schema follows normalization rules (1NF, 2NF, 3NF).

Sample Code:

```
CREATE TABLE Students (  
  student_id INT PRIMARY KEY,  
  name VARCHAR(100), email  
  VARCHAR(100)  
);  
CREATE TABLE Courses ( course_id  
  INT PRIMARY KEY, course_name  
  VARCHAR(100),  
  credits INT  
);  
CREATE TABLE Enrollments ( enrollment_id  
  INT PRIMARY KEY, student_id INT,  
  course_id INT,
```

FOREIGN KEY (student_id) REFERENCES

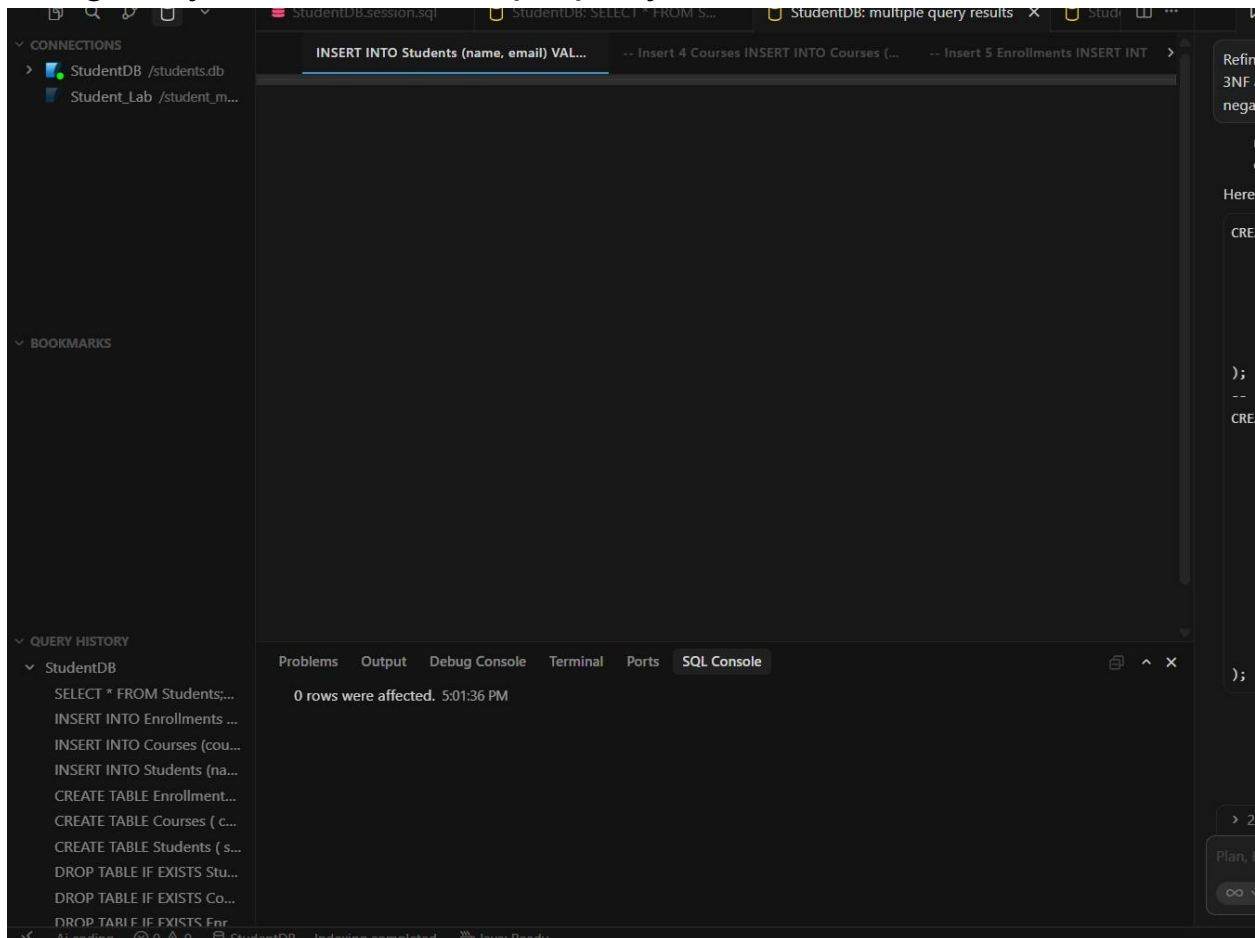
Students(student_id),

FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);

Expected Output:

Tables Students, Courses, and Enrollments are created successfully.

Foreign key constraints are properly enforced.



Task Description- 2: (Insert Sample Records) Instructions:

- Use AI tools to generate INSERT statements to populate the tables.
 - Insert at least 3 students, 4 courses, and 5 enrollment records.
- Verify data using SELECT queries.

Sample Code:

```
INSERT INTO Students VALUES
```

```
(1, 'Arjun', 'arjun@gmail.com'),
```

```
(2, 'Meera', 'meera@gmail.com'),
```

```
(3, 'Rahul', 'rahul@gmail.com');
```

```
INSERT INTO Courses VALUES
```

```
(101, 'Database Systems', 4),
```

```
(102, 'Data Structures', 3),
```

```
(103, 'Operating Systems', 4),
```

```
(104, 'AI Fundamentals', 3);
```

```
INSERT INTO Enrollments VALUES
```

```
(1, 1, 101),
```

```
(2, 1, 102),
```

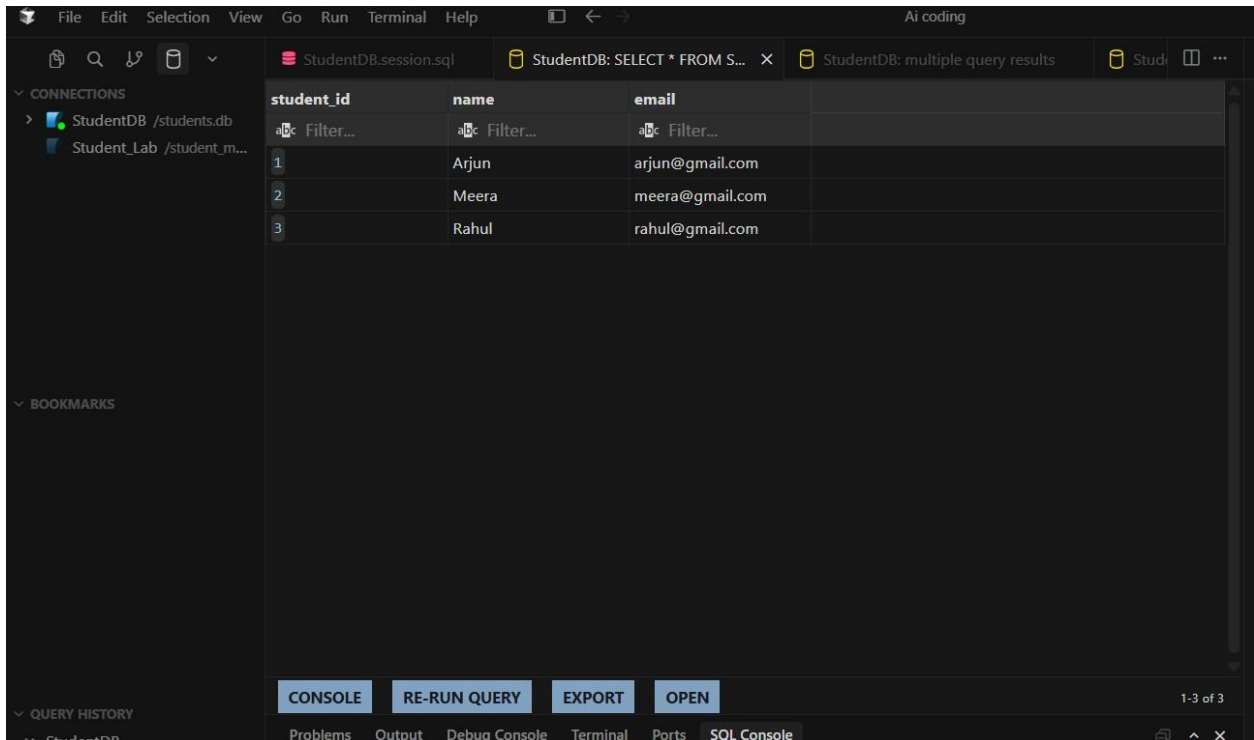
```
(3, 2, 103),
```

```
(4, 3, 101),
```

```
(5, 3, 104);
```

Expected Output:

- Sample records are inserted successfully.
- FROM Students; displays all student records.
- SELECT * FROM Enrollments; displays all enrollments.



Task Description- 3: (Perform CRUD Operations) Instructions:

Use AI to generate SQL queries for basic CRUD operations:

- Retrieve all courses
- Update a student's email
- Delete an enrollment record
- Validate the changes using SELECT statements.

Sample Code:

SELECT * FROM Courses;

UPDATE Students

SET email = 'arjun_new@gmail.com'

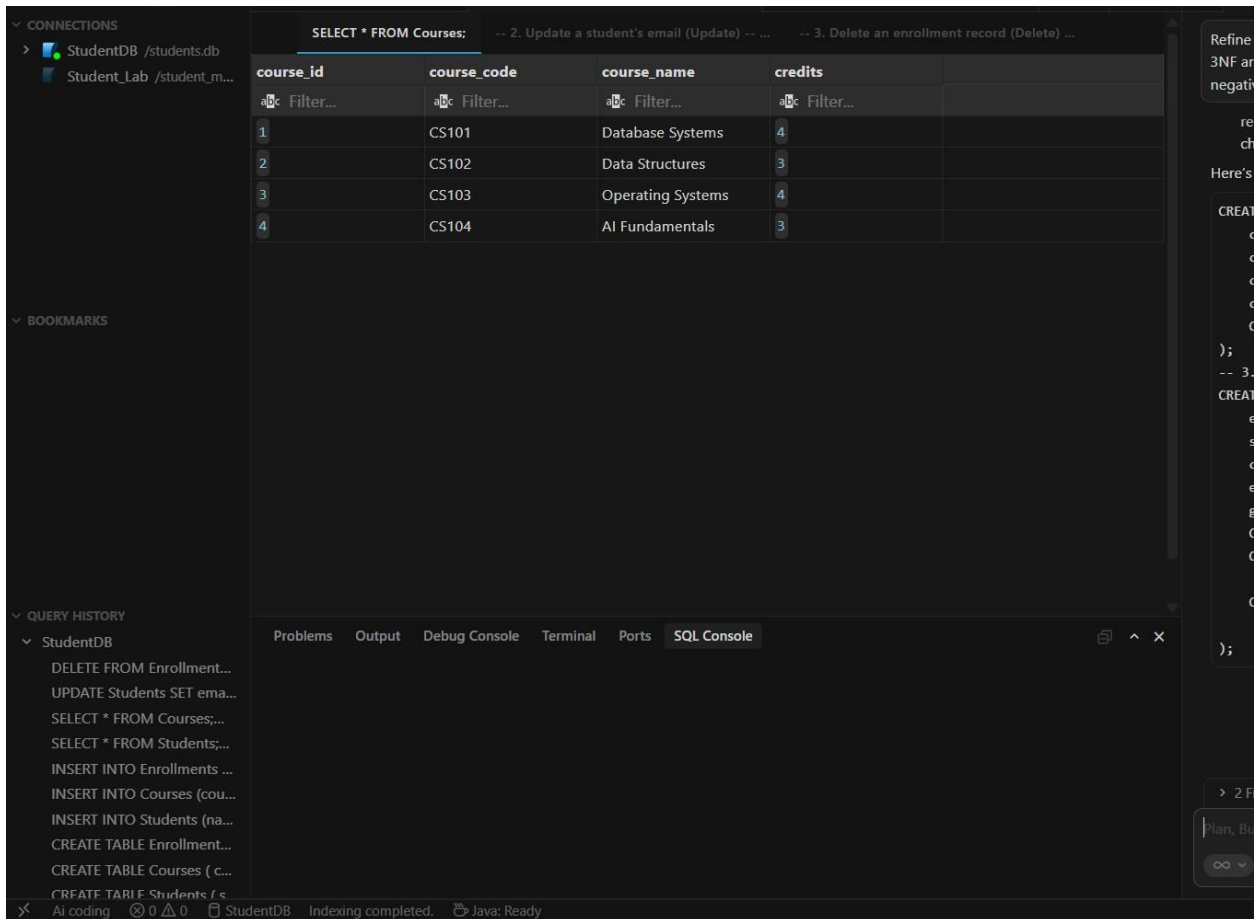
WHERE student_id = 1;

DELETE FROM Enrollments

WHERE enrollment_id = 5; Expected

Output:

- Updated email is reflected in Students table.
- Deleted enrollment record is removed successfully.
- Remaining data remains unchanged.



Task Description- 4: (Execute Aggregate Queries) Instructions:

Use AI tools to write aggregate queries:

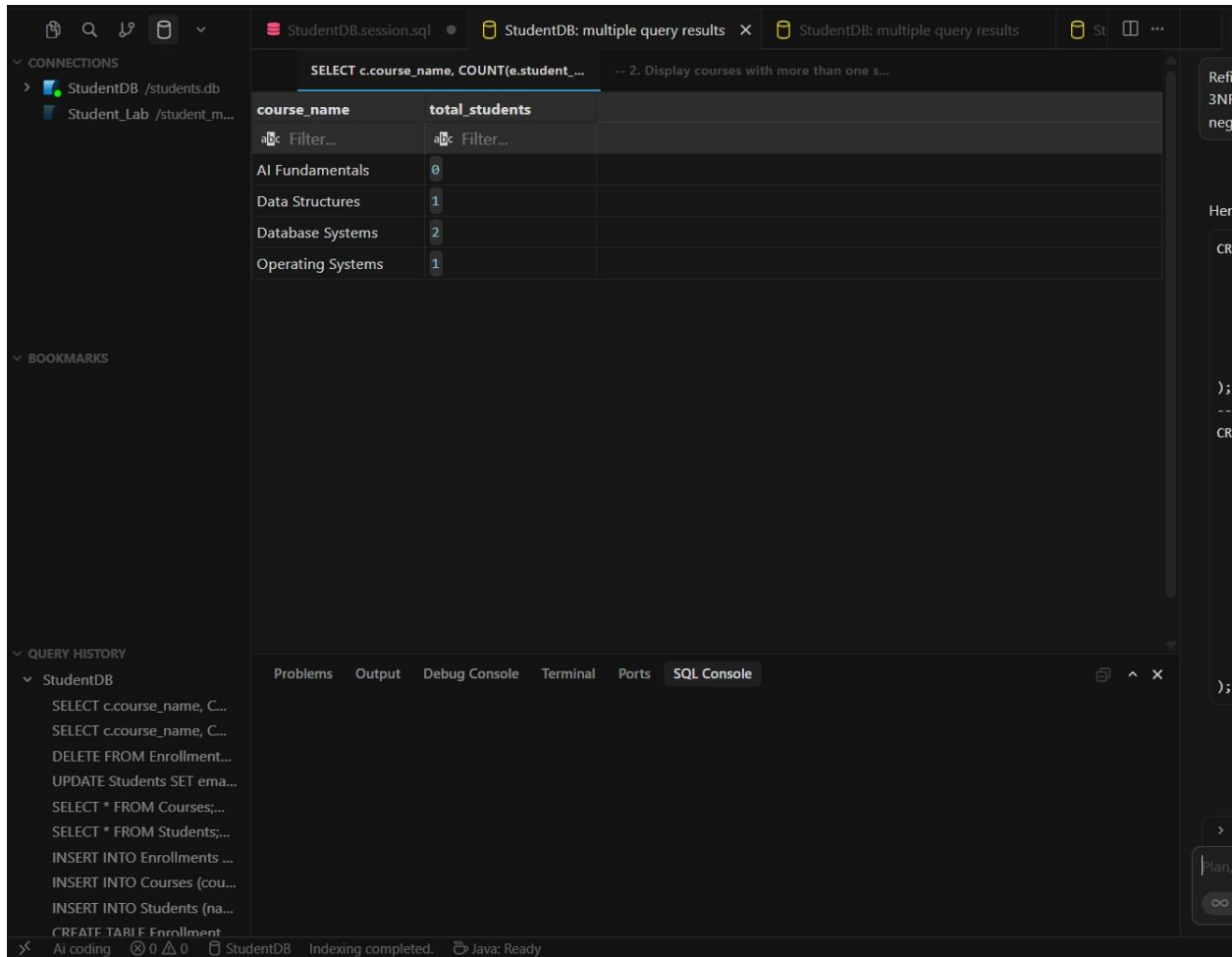
- Count number of students enrolled per course
 - Display courses with more than one student enrolled
- Analyze the correctness of results.

Sample Code:

```
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;
SELECT c.course_name
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
```

GROUP BY c.course_name HAVING
COUNT(e.student_id) > 1; Expected
Output:

- Correct count of students per course is displayed.
- Courses with multiple enrollments are listed accurately.



The screenshot shows a database IDE interface. The main window displays the results of a SQL query. The query is: `SELECT c.course_name, COUNT(e.student_id) FROM Courses c JOIN Enrollments e ON c.course_id = e.course_id GROUP BY c.course_name HAVING COUNT(e.student_id) > 1;`. The results are shown in a table with two columns: `course_name` and `total_students`. The table contains the following data:

course_name	total_students
AI Fundamentals	0
Data Structures	1
Database Systems	2
Operating Systems	1

The IDE also shows a sidebar with 'CONNECTIONS' and 'BOOKMARKS', and a 'QUERY HISTORY' panel at the bottom left. The status bar at the bottom indicates 'Indexing completed' and 'Java: Ready'.

Task Description- 5: (Query Optimization) Instructions:

- Use AI tools to analyze query performance.
- Rewrite inefficient sub queries using JOINS and test both queries for correctness.

Sample Code:

-- Original Query

```
SELECT * FROM Students
WHERE student_id IN (
```

```
SELECT student_id FROM Enrollments WHERE course_id = 101  
);
```

-- Optimized Query

```
SELECT s.*
```

```
FROM Students s
```

```
JOIN Enrollments e ON s.student_id = e.student_id
```

```
WHERE e.course_id = 101; Expected Output:
```

- Both queries return the same set of students.
- Optimized query performs better on large datasets.

This lab allows students to use AI tools like Copilot, Cursor, or Gemini to:

- Generate schema definitions
- Auto-complete insert statements
- Write CRUD and aggregate SQL queries
- Optimize queries and ensure proper normalization

File Edit Selection View Go Run Terminal Help Ai coding

StudentDB: multiple query results StudentDB: multiple query results StudentDB: REATE TABLE IF N...

CONNECTIONS

- StudentDB /students.db
- Student_Lab /student_m...

BOOKMARKS

student_id	name	email
1	Arjun	arjun_new@gmail.com
3	Rahul	rahul@gmail.com

CONSOLE RE-RUN QUERY EXPORT OPEN 1-2 of 2

QUERY HISTORY

CONNECTIONS

- StudentDB /students.db
- Student_Lab /student_m...

BOOKMARKS

student_id	name	email
1	Arjun	arjun_new@gmail.com
3	Rahul	rahul@gmail.com

CONSOLE RE-RUN QUERY EXPORT OPEN 1-2 of 2

QUERY HISTORY

StudentDB

Problems Output Debug Console Terminal Ports SQL Console